Lec 2: Architectures

Naveen Kumar

Assistant Professor
SVNIT, Surat

January 20, 2026

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

What Is Architecture in Distributed Systems?

- Distributed systems are complex because software runs on **multiple machines**
- This complexity must be managed through **proper organization**
- **Architecture** provides a structured way to design and understand such systems
- Architecture is viewed in two ways:
  - **Logical organization**
    - What components exist?
    - How components communicate?
  - **Physical realization**
    - Where components run?
    - How they are distributed across machines?

Software Architecture and Middleware

- From the architectural views discussed earlier, we now focus on the **logical organization** of a distributed system
- This logical organization is known as the **software architecture**
- Software architecture defines:
  - What software components exist
  - How these components interact with each other
- A key architectural goal is to keep applications **independent of physical deployment details**
- This separation is achieved using a **middleware layer**
- Middleware hides distribution aspects such as location, communication, and heterogeneity
- Complete transparency is not always desirable and involves **design trade-offs**

From Software Architecture to System Architecture

- So far, we focused on the **logical organization** of software components
- To build a real distributed system, this logical design must be **mapped onto physical machines**
- This involves:
  - Instantiating software components
  - Placing these components on actual machines
- The resulting physical deployment is called the **system architecture**
- Common system architectures include:
  - **Centralized architectures**: most functionality resides on one or few servers
  - **Decentralized (peer-to-peer) architectures**: nodes have roughly equal roles
  - **Hybrid architectures**: combine centralized and decentralized elements
- These concepts are illustrated using real-world examples such as **NFS** and **the Web**

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Architectural Styles: Software Architecture

- Architectural styles describe the **logical organization** of a distributed system called **software architecture**
- Software architecture focuses on:
  - Dividing the system into **software components**
  - Defining how these components **interact**
- A well-chosen architecture is essential for building **large, complex distributed systems**

Architectural Styles: Components and Types

- An **architectural style** specifies:
  - **Components**: modular units with well-defined interfaces
  - **Connectors**: communication mechanisms between components
  - Rules for composing components into a system
- Common architectural styles in distributed systems:
  - Layered
  - Object-based
  - Resource-centered (REST)
  - Event-based (publish–subscribe)
- Real systems often **combine multiple styles**, with layering being almost universal

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Layered Architectures: Basic Principle

- A layered architecture organizes components into a hierarchy of layers
- Higher layers use services of lower layers through **downcalls**
- Direct calls to higher layers are generally avoided
- This structure separates responsibilities and improves maintainability

Variants of Layered Architectures

- Layered architectures have **three common variants**
- **Pure layering**: each layer communicates only with the layer directly below
- **Mixed layering**: a layer may access multiple lower layers
- Pure layering is common in protocol stacks, while mixed layering is used in OS and libraries

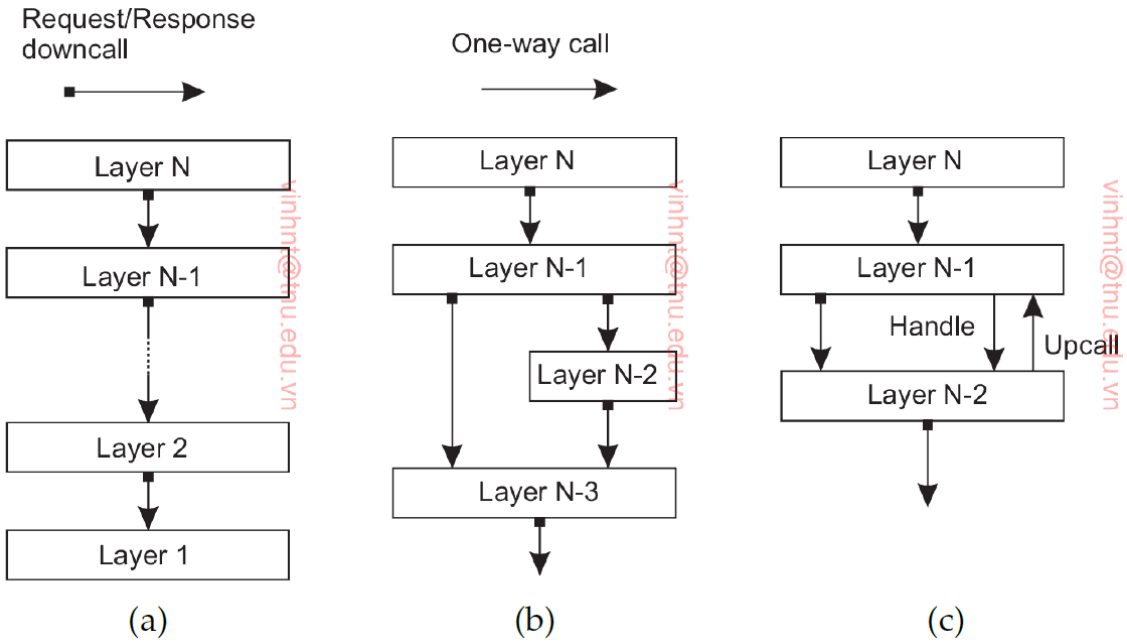


Figure 1: (a) Pure layering (b) Mixed layering (c) Upcalls

Layered Protocol Stacks: Service, Interface, Protocol

- Each layer is described using three related concepts:
  - **Service**: what the layer provides to higher layers
  - **Interface**: how the service is accessed
  - **Protocol**: how the service is implemented between peer layers
- The interface hides implementation details, while the protocol defines communication rules
- This separation helps structure communication systems into manageable parts

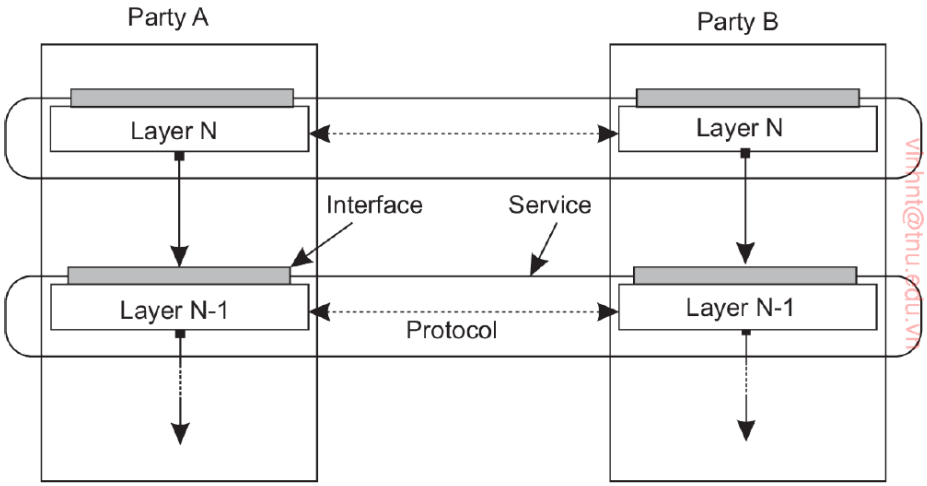


Figure 2: Layered protocol stack: service, interface, and protocol

Example: Reliable, Connection-Oriented Communication

- A widely used communication service is **reliable and connection-oriented**
- Before data transfer, a **connection is established**
- The service guarantees:
  - Reliable message delivery
  - Preservation of message order
- On the Internet, this service is provided by **TCP**
- The protocol defines connection setup, error handling, and data ordering

Example: Connection-Oriented Server (Sockets)

```
from socket  import *
s = socket(AF_INET, SOCK_STREAM)
(conn, addr) = s.accept()  # returns new socket and addr. client
while True:                # forever
  data = conn.recv(1024)   # receive data from client
  if not data: break       # stop if client stopped
  conn.send(str(data)+"*") # return sent data plus an "*"
conn.close()               # close the connection
```

(a) A simple server

```
from socket  import *
s = socket(AF_INET, SOCK_STREAM)
s.connect((HOST, PORT)) # connect to server (block until accepted)
s.send('Hello, world')  # send some data
data = s.recv(1024)     # receive the response
print data              # print the result
s.close()               # close the connection
```

(b) A client

Figure 2.3: Two communicating parties.

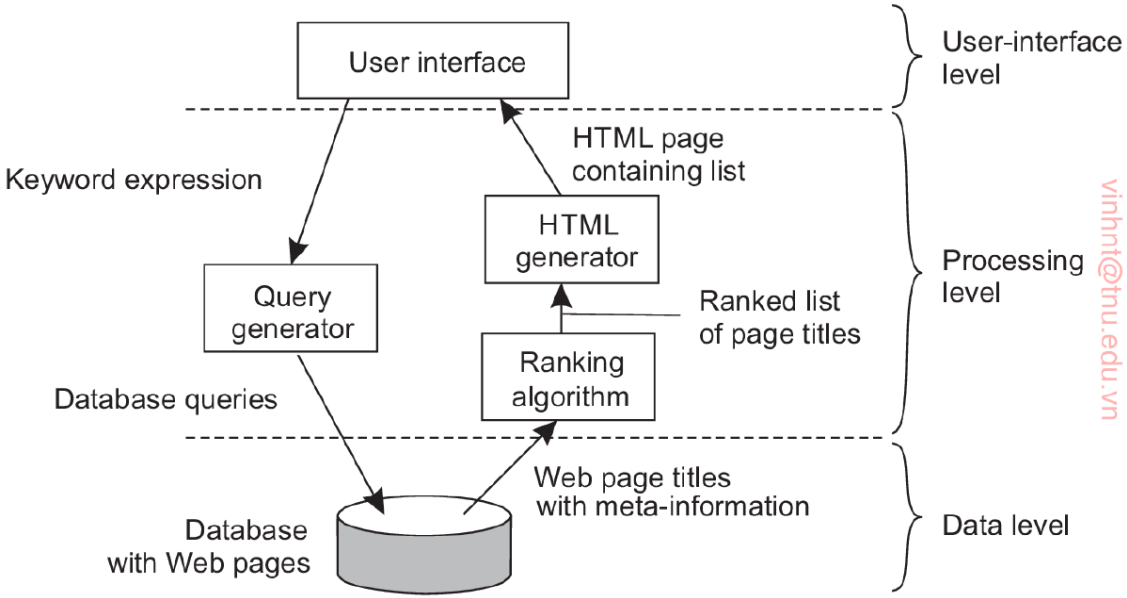
- Applications access connection-oriented services using **sockets**
- The socket interface exposes the underlying TCP service
- Typical server-side operations include:
  - `socket()` – create a communication endpoint
  - `accept()` – accept client connections
  - `send()`, `recv()` – exchange data
  - `close()` – terminate the connection

Application Layering: Concept and Purpose

- Many distributed applications use **logical layering**
- The goal is to clearly separate application responsibilities
- A common organization uses three layers:
  - **Interface layer**: user or API interaction
  - **Processing layer**: core application logic
  - **Data layer**: persistent data management
- This separation improves modularity, maintainability, and scalability

Application Layering: Example (Internet Search Engine)

- A search engine illustrates three-layer organization
- **Interface layer**:
  - Accepts user queries
  - Displays search results
- **Processing layer**:
  - Translates queries into database operations
  - Ranks and formats results
- **Data layer**:
  - Stores indexed web pages



vinhnt@tnu.edu.vn

Figure 3: Three-layer organization of an Internet search engine

Table of Contents

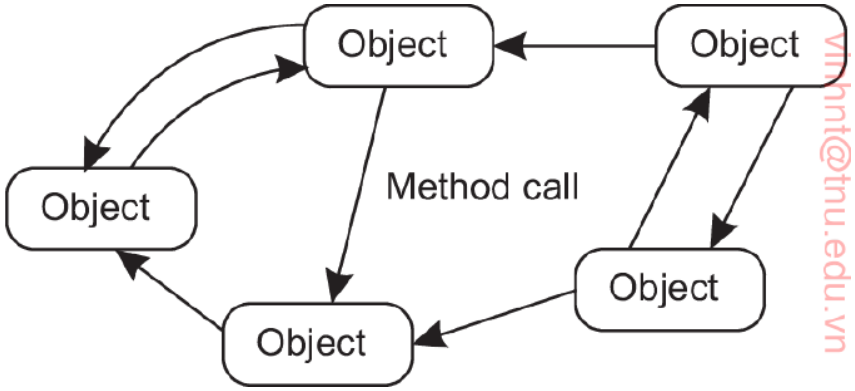
1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Object-Based Architectures: Core Idea

- Object-based architectures organize systems as a collection of **loosely coupled objects**
- Each object represents a **software component** with a clear responsibility
- Objects interact using **procedure calls**
- In distributed systems, these calls may cross machine boundaries
- This approach naturally supports modular system design

Encapsulation in Object-Based Architectures

- Each object encapsulates:
  - **State**: internal data
  - **Methods**: operations on that data
- Objects expose functionality only through **interfaces**
- Interfaces hide internal implementation choices
- Objects can be replaced as long as interfaces remain unchanged
- This simplifies system evolution and maintenance



vinhnt@tnu.edu.vn

Figure 4: Object-based architectural style

Distributed Objects

- In a **distributed object**, the object state resides on one machine
- Only the object's interface is accessible remotely
- Clients interact with the object through a **proxy**
- The proxy converts method calls into messages and back
- Server-side requests are handled by a **stub (skeleton)**
- Distribution details are hidden from the client

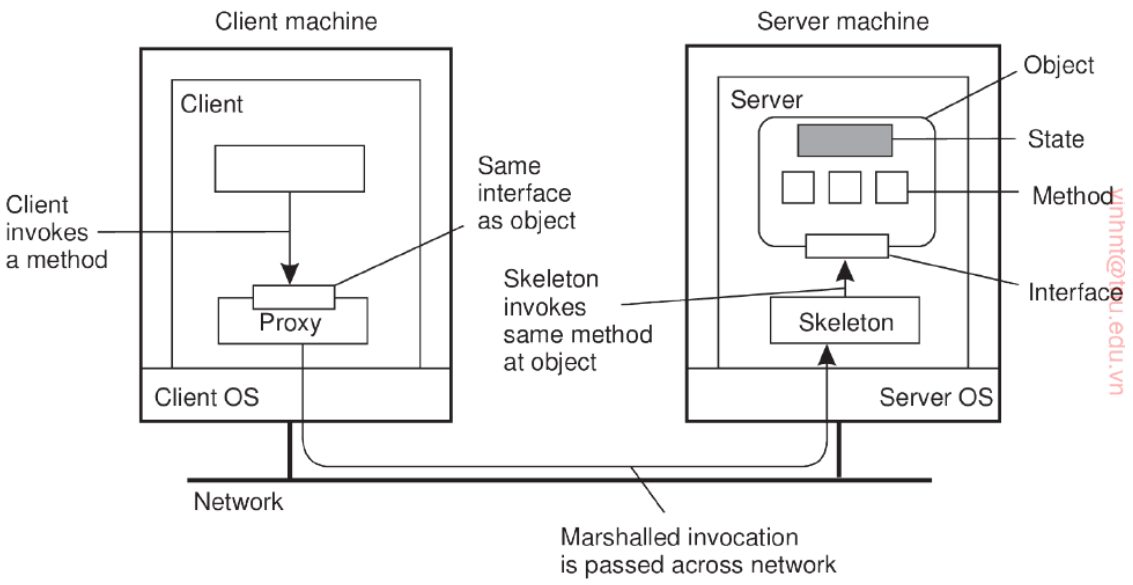


Figure 5: Remote object with client-side proxy

From Object-Based to Service-Oriented Architectures

- Encapsulated objects naturally evolve into **services**
- In a **Service-Oriented Architecture (SOA)**:
  - Systems are composed of independent services
  - Services may be owned by different organizations
- Example:
  - A web shop uses an external payment service
  - Payment processing is independent of order management
- Each service exposes a **well-defined interface**
- Service composition increases flexibility but also complexity

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Resource-Based Architectures: Motivation

- As Web services increased, **service composition** became common
- Composing many services often leads to:
  - Tight coupling
  - Complex integration
  - Difficult maintenance
- An alternative view:
  - A distributed system as a **collection of resources**
  - Resources are managed and accessed in a uniform way
- This idea forms the basis of **Resource-Based Architectures**

REST: Representational State Transfer

- Resource-based architectures are widely known as **REST**
- REST treats a distributed system as:
  - A set of **resources**
  - Each resource managed independently
- Resources can be:
  - Created
  - Retrieved
  - Modified
  - Deleted
- REST has been widely adopted for **Web-based systems**

Key Characteristics of RESTful Architectures

- RESTful systems have four defining properties:
  - **Uniform resource identification**
    - All resources use a single naming scheme (URIs)
  - **Uniform interface**
    - All services offer the same limited set of operations
  - **Self-descriptive messages**
    - Requests and responses contain all necessary information
  - **Stateless execution**
    - Services do not store information about previous requests

REST Operations and Example: Cloud Storage

- REST uses a small set of standard operations:
  - **PUT** – create a resource
  - **GET** – retrieve a resource
  - **POST** – modify a resource
  - **DELETE** – remove a resource
- Example: **Amazon S3**
  - Resources: buckets and objects
  - Objects are accessed using URIs
  - Operations are standard HTTP requests
- REST is simple and scalable, but:
  - Less suitable for complex interactions
  - Difficult to support distributed transactions

Operation	Description
PUT	Create a new resource
GET	Retrieve the state of a resource in some representation
DELETE	Delete a resource
POST	Modify a resource by transferring a new state

Figure 6: REST operations available on resources

Note 2.2: Interfaces vs Services (Advanced)

- A **service** provides functionality
- An **interface** only defines how that functionality is accessed
- An interface cannot make a service inherently:
  - Easier
  - More powerful
  - More complex
- Much of the debate between:
  - **RESTful interfaces**
  - **Service-specific interfaces**
- Is actually about **access transparency**, not functionality

Example: Amazon S3 — REST vs SOAP Interfaces

- Amazon S3 offers two different interfaces:
  - **REST interface**: very few generic operations
  - **SOAP interface**: many service-specific operations
- SOAP interface exposes:
  - ∼16 operations (see Figure 7)
  - Even more via libraries (e.g., ∼50 in Python boto)
- REST interface uses:
  - A small fixed set of operations (GET, PUT, POST, DELETE)
  - Functionality encoded in **parameters**

Bucket operations	Object operations
ListAllMyBuckets	PutObjectInline
CreateBucket	PutObject
DeleteBucket	CopyObject
ListBucket	GetObject
GetBucketAccessControlPolicy	GetObjectExtended
SetBucketAccessControlPolicy	DeleteObject
GetBucketLoggingStatus	GetObjectAccessControlPolicy
SetBucketLoggingStatus	SetObjectAccessControlPolicy

Figure 7: The operations in Amazon's S3 SOAP interface

Trade-offs in Interface Design

- **Service-specific interfaces**:
  - Clear operation semantics
  - Errors can be detected early (e.g., compile time)
  - Easier to understand intended behavior
- **Generic (RESTful) interfaces**:
  - All meaning encoded in parameters or strings
  - Errors detected at runtime
  - Easier to evolve and extend
- Key trade-off:
  - **Clarity and safety** vs **flexibility and adaptability**

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Publish–Subscribe Architectures: Motivation

- As distributed systems grow, processes frequently:
  - Join and leave dynamically
  - Belong to different organizations
- Tight dependencies between processes become problematic
- A key architectural goal is **loose coupling**
- Many systems separate:
  - **Processing**: what each process does
  - **Coordination**: how processes communicate and cooperate
- Coordination acts as the **glue** that binds independent processes

Coordination Models in Distributed Systems

- Coordination models differ along two dimensions:
  - **Temporal coupling**:
    - Whether sender and receiver must be active at the same time
  - **Referential coupling**:
    - Whether processes explicitly know each other
- This leads to four coordination styles:
  - **Direct coordination**: temporally and referentially coupled
  - **Mailbox coordination**: temporally decoupled, referentially coupled
  - **Event-based coordination**: referentially decoupled, temporally coupled
  - **Shared data space**: temporally and referentially decoupled

	Temporally coupled	**Temporally decoupled**
Referentially coupled	Direct	Mailbox
Referentially decoupled	Event-based	Shared data space

Figure 8: Examples of different forms of coordination.

Publish–Subscribe Architectures

- In **publish–subscribe** systems:
  - Processes do not explicitly reference each other
  - Publishers announce events
  - Subscribers express interest in certain events
- Event-based coordination:
  - Subscribers must be active when events are published
- Shared data spaces:
  - Processes communicate via shared data (e.g., tuples)
  - Data can be retrieved later using associative matching
- Both styles rely on **decoupling** and are forms of publish–subscribe

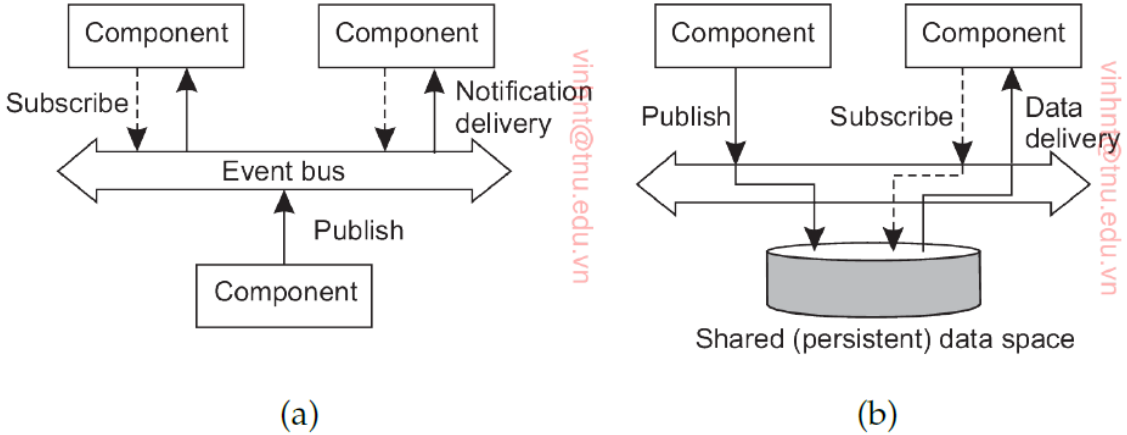


Figure 9: The (a) event-based and (b) shared data-space architectural style.

Subscriptions, Matching, and Challenges

- Subscribers describe events of interest using:
  - **Topic-based subscriptions** (attribute–value pairs)
  - **Content-based subscriptions** (attribute ranges or predicates)
- Middleware matches:
  - Published notifications
  - Against stored subscriptions
- Two delivery models:
  - Data sent directly with the notification
  - Notification sent; data retrieved separately
- Major challenges:
  - Efficient and scalable matching
  - Event composition from distributed sources

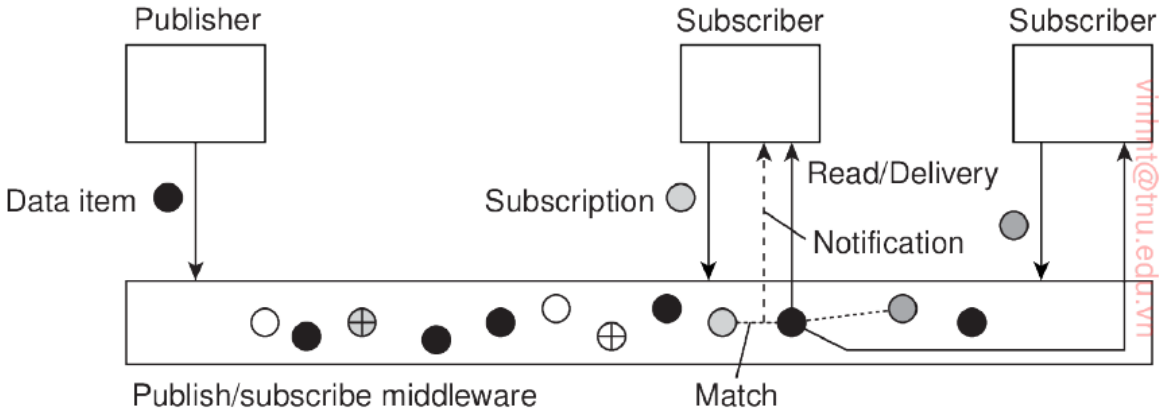


Figure 2.12: The principle of exchanging data items between publishers and subscribers.

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Middleware Organization: From Architecture to Design Patterns

- Earlier, we discussed **architectural styles** for organizing distributed systems
- We now focus on the **internal organization of middleware**
- This discussion is independent of:
  - Application structure
  - Overall system architecture
- Two common middleware design patterns:
  - **Wrappers**
  - **Interceptors**
- Both aim to achieve **openness** in middleware
- Ultimate openness is achieved when middleware can be:
  - **Composed and modified at runtime**
- **Component-based middleware** enables such modifiable middleware

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Wrappers: The Interface Compatibility Problem

- Distributed systems are often built from **existing (legacy) components**
- These components expose **native interfaces**
- Native interfaces are often:
  - Not designed for new applications
  - Incompatible with each other
- Direct reuse through native interfaces is therefore difficult
- This creates a major obstacle to system integration

Wrappers: Core Idea

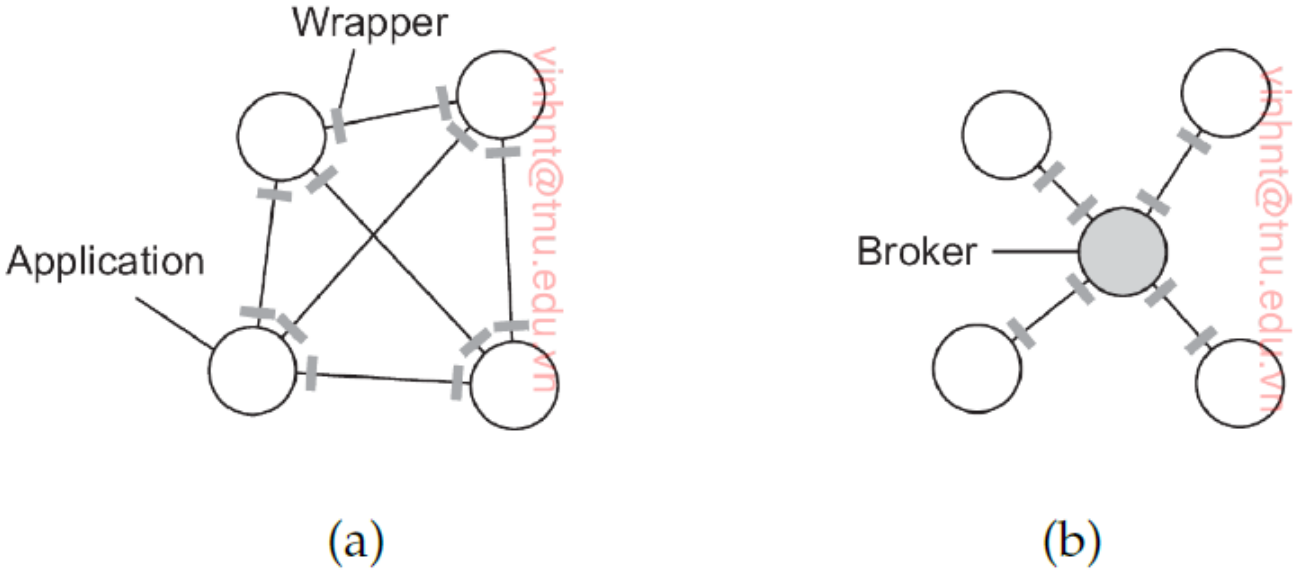
- A **wrapper (adapter)** is a special component
- It provides an interface that is:
  - Acceptable to the client application
  - Different from the native interface of the component
- The wrapper:
  - Translates client requests
  - Invokes corresponding operations on the underlying component
- Wrappers solve the problem of **incompatible interfaces**
- This idea originates from object-oriented design patterns

Wrappers in Distributed Systems

- In distributed systems, **wrappers** hide distribution and implementation details
- They expose a **clean, consistent interface** to applications
- Example: **Object adapters**
  - Enable access to remote objects
  - Underlying implementation may be libraries or databases
- **Amazon S3 example**:
  - Clients use a RESTful HTTP interface
  - A Web server acts as a wrapper, translating requests to internal S3 operations

Scalability Problem of Wrappers

- Wrappers enable extensibility and openness
- Naïve approach:
  - Each application builds a wrapper for every other application
- With N applications:
  - Number of wrappers grows as $N(N-1)$
  - Complexity is $O(N^2)$
- This approach does **not scale** for large systems



vinhnt@tnu.edu.vn

Figure 11: (a) Requiring each application to have a wrapper for each other application. (b) Reducing the number of wrappers by making use of a broker.

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Interceptors: Concept and Purpose

- An **interceptor** is a hook placed in the middleware execution path
- It allows extra code to run **before or after** a normal operation
- Used to **modify or extend middleware behavior** without changing applications
- Helps make middleware more **flexible and adaptable**
- In practice, only limited interception is used to keep systems:
  - Easy to manage
  - Reliable

Interceptors in Remote Object Invocation

- Remote object invocation proceeds in three steps:
  - Object A calls a local interface of object B
  - Call is transformed into a **generic invocation**
  - Invocation is converted into a network message
- **Request-level interceptors** can:
  - Modify or replicate invocations
  - Handle object replication transparently
- **Message-level interceptors** can:
  - Fragment large data
  - Improve performance or reliability
- Neither applications nor middleware core need to be aware

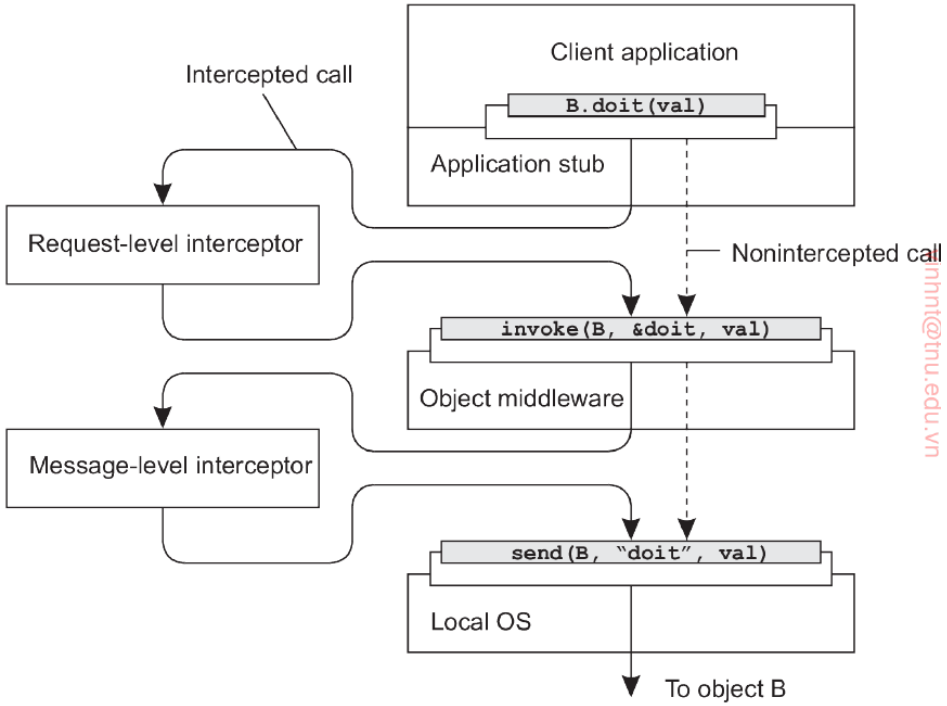


Figure 12: Using interceptors in remote-object invocation

linhnt@tnu.edu.vn

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Modifiable Middleware: Motivation

- Distributed systems run in **dynamic and changing environments**
- Typical changes include:
  - Mobile users and devices
  - Varying network speed and reliability
  - Hardware failures and limited battery power
- Handling these changes inside applications is **difficult and error-prone**
- Therefore, this responsibility is shifted to the **middleware**
- Large distributed systems cannot be stopped for upgrades
- Hence, middleware must support **runtime (on-the-fly) adaptation**

Modifiable Middleware: How It Is Achieved

- **Modifiable middleware** allows controlled changes while the system is running
- One key mechanism is the use of **interceptors**
  - They adapt or extend middleware behavior dynamically
- Another approach is **replacing or updating components at runtime**
- Most modifiable middleware follows a **component-based design**
- Middleware is built by composing components:
  - **Static composition**: fixed at design time
  - **Dynamic composition**: changed during execution
- Runtime modification requires:
  - Loading and unloading components safely
  - Clearly defined interfaces between components
  - Careful handling of components that maintain state

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

System Architecture

- So far, we discussed **architectural styles** for organizing software
- We now focus on how distributed systems are **actually deployed**
- **System architecture** concerns:
  - Which software components are used
  - How these components interact
  - Where components are placed on physical machines
- A concrete placement of components forms a **system architecture**
- Common system organizations:
  - Centralized architectures
  - Decentralized architectures
  - Hybrid architectures

Table of Contents

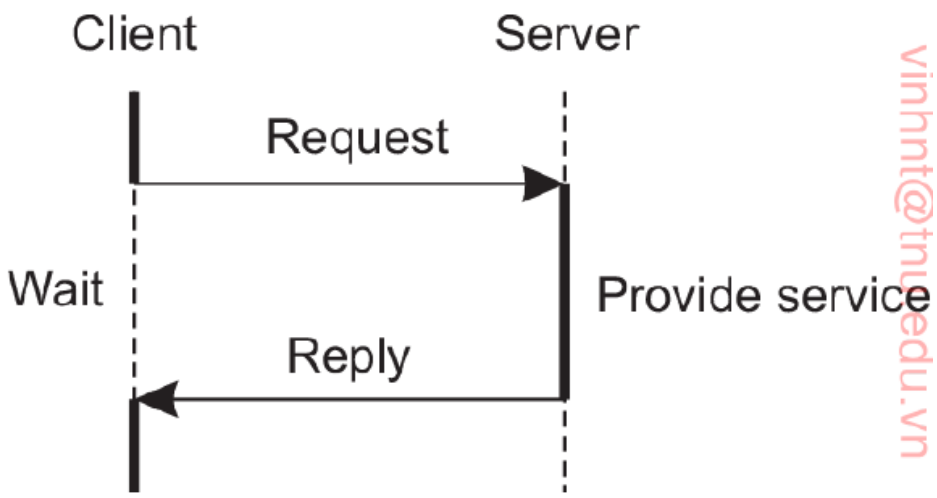
1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Centralized Organizations

- A widely accepted model for distributed systems is the **client–server paradigm**
- Clients **request services** from one or more servers
- This model helps:
  - Understand system structure
  - Manage system complexity
- Centralized thinking simplifies:
  - Control
  - Management
  - Reasoning about behavior
- We first consider:
  - **Simple layered organizations**
  - Then **multi-layered organizations**

Simple Client–Server Architecture

- Distributed systems often follow the **client–server model**
- Processes are divided into two roles:
  - **Server**: implements a specific service
  - **Client**: requests that service
- Interaction follows **request–reply behavior**:
  - Client sends a request
  - Server processes the request
  - Server sends a reply
- This interaction is the basic building block of centralized systems



vinhnt@tnuedu.vn

Figure 13: General interaction between a client and a server

Client–Server Communication and Limitations

- Communication can be implemented using:
  - **Connectionless protocols**
    - Efficient in reliable local networks
    - Message loss may cause duplicate execution
    - Safe only for **idempotent operations**
  - **Connection-oriented protocols** (e.g., TCP)
    - Reliable communication
    - Widely used in Internet applications
    - Higher overhead due to connection setup
- Client and server roles are not always strict:
  - A server may act as a client to other servers

Multitiered Architectures: Two-Tier Model

- Applications are often divided into three logical layers:
  - User interface layer
  - Processing layer
  - Data layer
- These layers can be **physically distributed** across machines
- The simplest organization is a **two-tier architecture**:
  - **Client machine**: user-interface software
  - **Server machine**: processing and data management
- In its simplest form, the client acts as a **thin terminal**

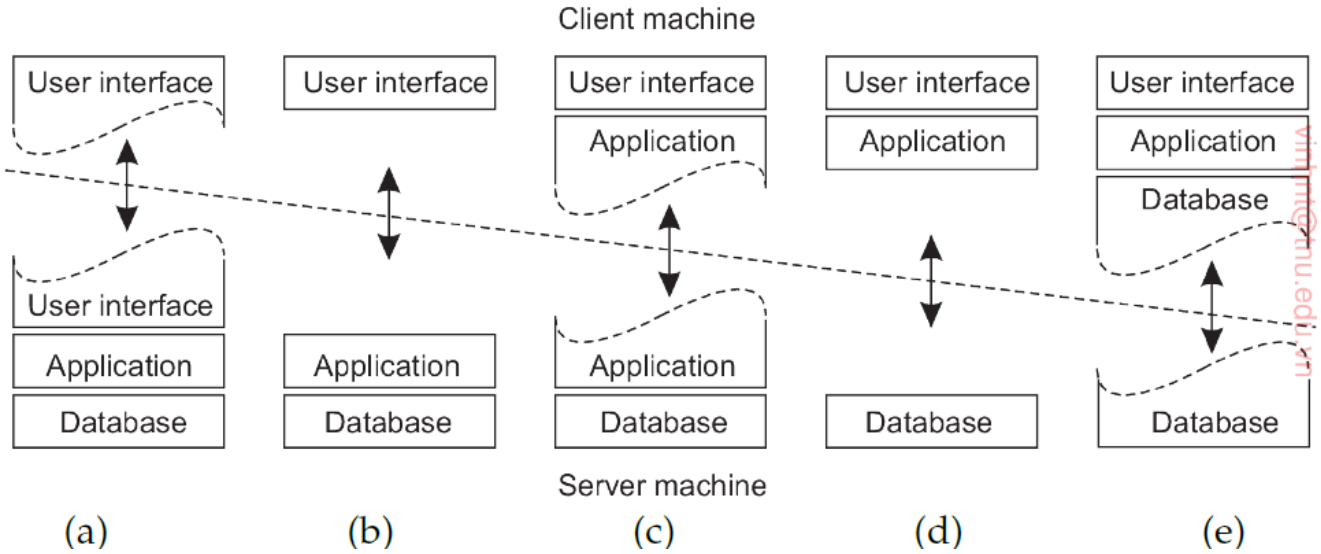


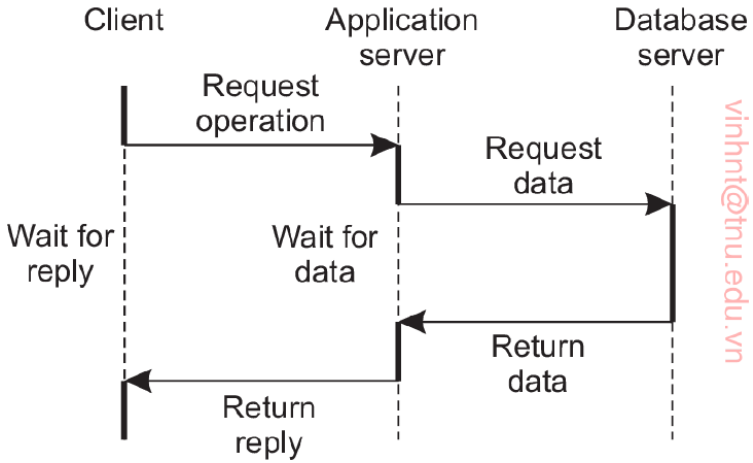
Figure 14: Client-server organizations in a two-tiered architecture.

Two-Tier Variants: Client–Server Placement

- Different two-tier organizations differ in how much work the client does
- Common variants:
  - Only terminal-dependent UI on client (remote presentation control)
  - Full user interface on client; server handles all processing
  - Part of application logic on client (e.g., form validation)
- Examples:
  - Word processor: basic editing on client, advanced tools on server
  - Banking apps: transaction preparation on client, execution on server
- Client may also cache data locally (e.g., web browser cache)

Three-Tier Architectures and Client Fatness

- In practice, servers may also act as **clients**
- This leads to a **three-tier architecture**:
  - Client
  - Application (processing) server
  - Data server
- Common examples:
  - Transaction processing systems with a transaction monitor
  - Web systems:
    - Web server (entry point)
    - Application server (business logic)
    - Database server (data storage)
- Trend:
  - Thin clients are easier to manage
  - Fat clients remain useful for rich and multimedia applications



vinhnt@tnu.edu.vn

Figure 15: An example of a server acting as client

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Decentralized Organizations: Distribution Models

- Multitier client–server architectures rely on **vertical distribution**
  - Logically different components placed on different machines
  - User interface, processing, and data are separated
- Vertical distribution helps simplify system management
- An alternative approach is **horizontal distribution**
  - Clients or servers are split into **logically equivalent parts**
  - Each part handles a share of the overall data
  - Load is balanced across multiple machines
- Architectures supporting horizontal distribution are known as **peer-to-peer (P2P) systems**

Peer-to-Peer Systems: Core Characteristics

- In peer-to-peer systems:
  - All processes are **equal**
  - Each process acts as both **client and server**
  - Interaction between processes is largely symmetric
- Processes are organized in an **overlay network**
  - Nodes represent processes
  - Links represent possible communication channels
- Overlay networks can be classified as:
  - **Structured**
  - **Unstructured**

Structured Peer-to-Peer Systems

- In **structured P2P systems**, nodes are organized using a **deterministic overlay topology**
- Common topologies:
  - Ring
  - Tree
  - Grid
  - Hypercube
- The topology is designed to support **efficient data lookup**
- Data placement and routing are fully determined by structure

Distributed Hash Tables (DHTs)

- Structured P2P systems use a **semantic-free index**
- Each data item is mapped to a key:

$$key = hash(\text{data item})$$

- The system stores **(key, value)** pairs
- Each node:
  - Has a unique identifier
  - Is responsible for a subset of keys
- This forms a **Distributed Hash Table (DHT)**

- Fundamental operation:

$$node = lookup(key)$$

- Any node can initiate a lookup
- Overlay topology ensures:
  - Correct routing
  - Efficient lookup

Example: Hypercube-Based Peer-to-Peer System

- Consider a peer-to-peer system organized as a **hypercube**
- Nodes have binary identifiers
- Data items are hashed to keys matching node identifiers
- Lookup works by:
  - Forwarding requests to a neighbor
  - That is closer to the target key
- Each hop reduces the distance to the destination node

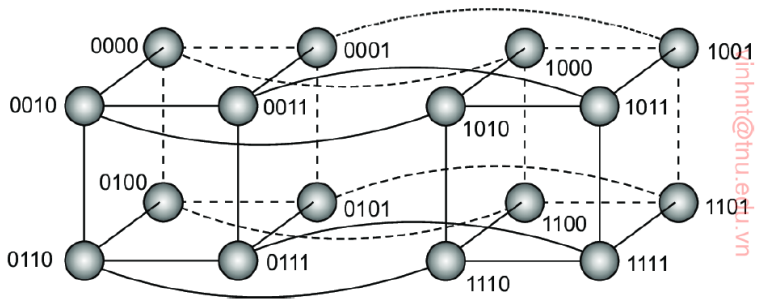


Figure 16: Four-dimensional hypercube overlay

linhnt@tnu.edu.vn

Example: The Chord System

- Chord organizes nodes in a **logical ring**
- Keys and node identifiers use the same identifier space
- A key k is stored at:

  $succ(k) =$ first node with $id \geq k$

- Each node maintains **shortcuts (finger table)**
- Lookup complexity:
  - $O(\log N)$ hops for N nodes

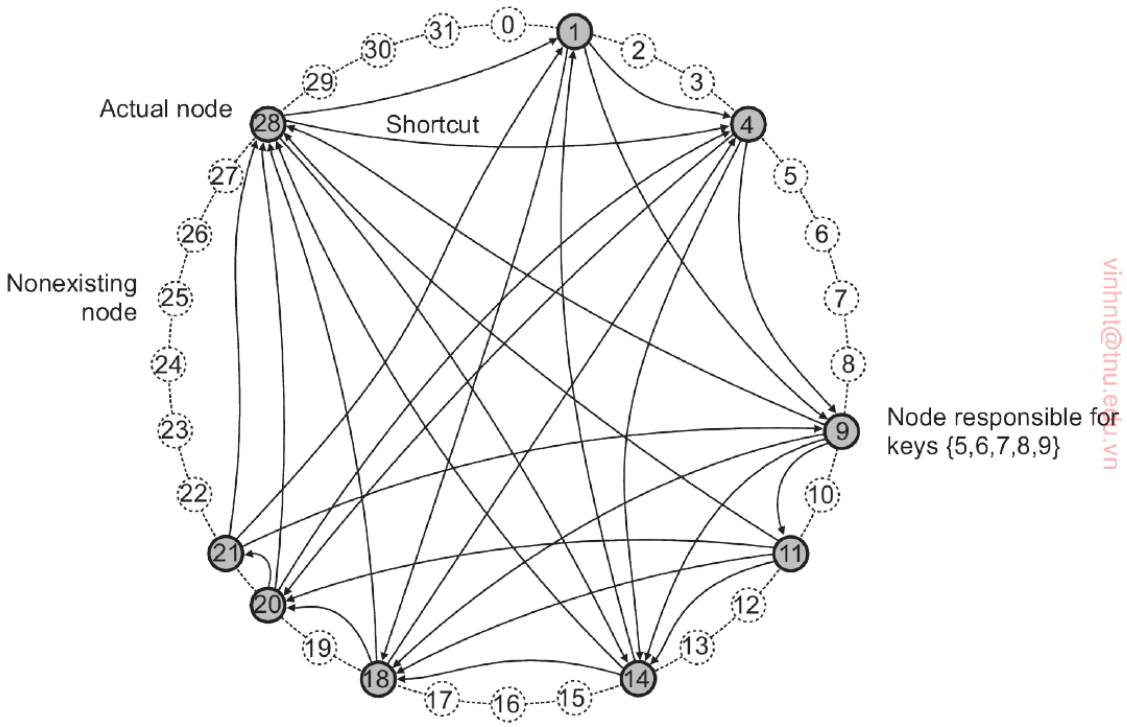


Figure 17: Node and key placement in Chord

vinhnt@tnu.edu.vn

Unstructured Peer-to-Peer Systems

- Unstructured P2P systems do **not maintain a fixed overlay topology**
- Each node keeps an **ad hoc list of neighbors**
- The resulting overlay resembles a **random graph**
- Neighbor relationships are **dynamic**:
  - Nodes join and leave frequently
  - Unresponsive neighbors are replaced
- Nodes usually discover peers by:
  - Contacting a well-known node
  - Gradually learning about other peers

Searching in Unstructured P2P: Flooding

- Without structure, data lookup requires **search**
- **Flooding** is the simplest approach:
  - A node forwards a request to **all its neighbors**
  - Each neighbor repeats this process
  - Requests already seen are ignored
- To limit cost, requests use a **time-to-live (TTL)**:
  - TTL too small $\rightarrow$ data may not be found
  - TTL too large $\rightarrow$ excessive network traffic
- Flooding finds data quickly but is **expensive**

Searching in Unstructured P2P: Random Walks

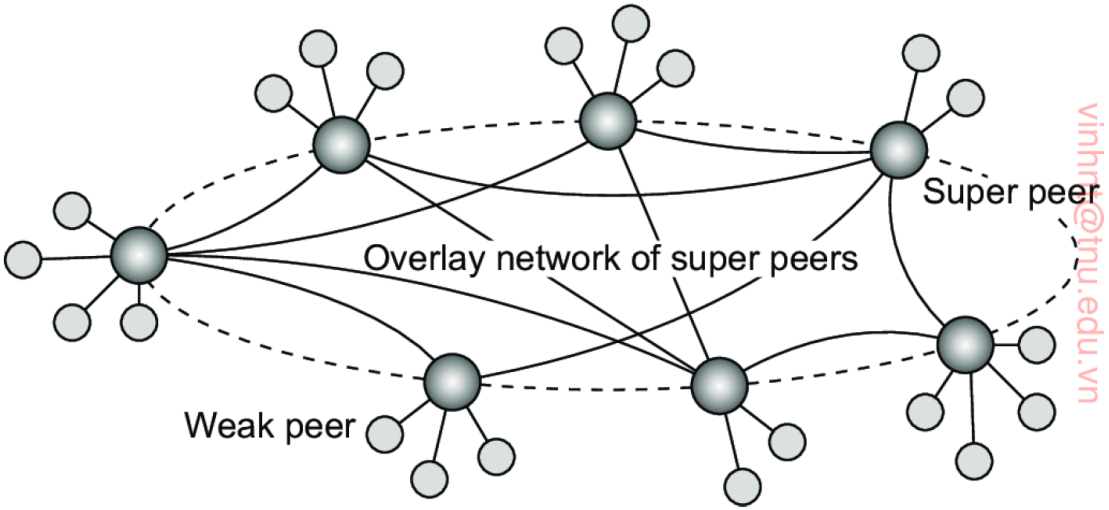
- An alternative to flooding is a **random walk**
- A node forwards the request to **one randomly chosen neighbor**
- That neighbor repeats the process
- Advantages:
  - Much lower network traffic
  - Scales better than flooding
- Disadvantages:
  - May take longer to find data
- Performance can be improved by:
  - Launching multiple random walks in parallel
  - Using policy-based or hybrid search strategies

Hierarchically Organized Peer-to-Peer Networks

- In large **unstructured P2P systems**, locating data becomes difficult
- Reason:
  - No deterministic routing
  - Searches rely on flooding or random walks
  - Poor scalability as the network grows
- A common solution is to introduce **special nodes**
  - Maintain indexes of data items
  - Collect information about resources and availability
- This relaxes the strict symmetry of pure P2P systems

Super Peers and Hierarchical Organization

- Special nodes are called **super peers**
- Regular nodes are referred to as **weak peers**
- Characteristics:
  - Weak peers connect to a super peer as clients
  - All communication from a weak peer goes via its super peer
- Super peers often:
  - Maintain indexes of data or users
  - Act as brokers for resource selection
- Super peers themselves are often organized in a **P2P network**



vinhnr@tnu.edu.vn

Figure 18: A hierarchical organization of nodes into a super-peer network

Dynamics and Example: The Skype Network

- Association between weak peers and super peers can be:
  - Fixed (until a peer leaves)
  - Dynamic (switching to better super peers)
- Super peers are expected to be:
  - Long-lived
  - Highly available
- Selecting super peers introduces a new challenge:
  - Closely related to **leader election**
- **Skype** is a well-known example:
  - Uses super peers for lookup and NAT traversal
  - Includes a centralized login server
  - Combines decentralization with limited central control

Table of Contents

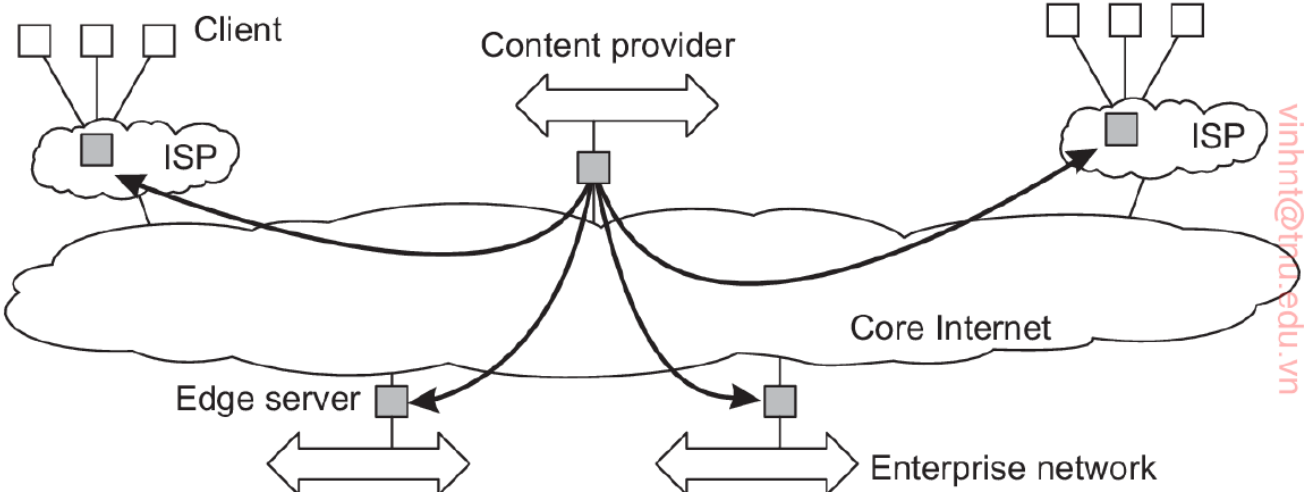
1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Hybrid Architectures

- So far, we have studied:
  - **Client–server architectures**
  - **Peer-to-peer architectures**
- Many real-world distributed systems **combine architectural features**
- Such systems are known as **hybrid architectures**
- Hybrid systems integrate:
  - Centralized components (e.g., servers, brokers)
  - Decentralized components (e.g., peer-to-peer overlays)
- **Super-peer networks** are an early example of hybrid design
- Hybrid architectures balance:
  - Scalability
  - Performance
  - Manageability

Edge-Server Systems: Concept

- Edge-server systems are an important form of **hybrid architecture**
- Servers are placed at the **edge of the Internet**
  - Boundary between enterprise networks and the public Internet
  - Often operated by Internet Service Providers (ISPs)
- End users typically access the Internet via an **edge server**
- Edge servers act as an intermediary between:
  - Clients
  - Core Internet infrastructure



vinhnt@tnu.edu.vn

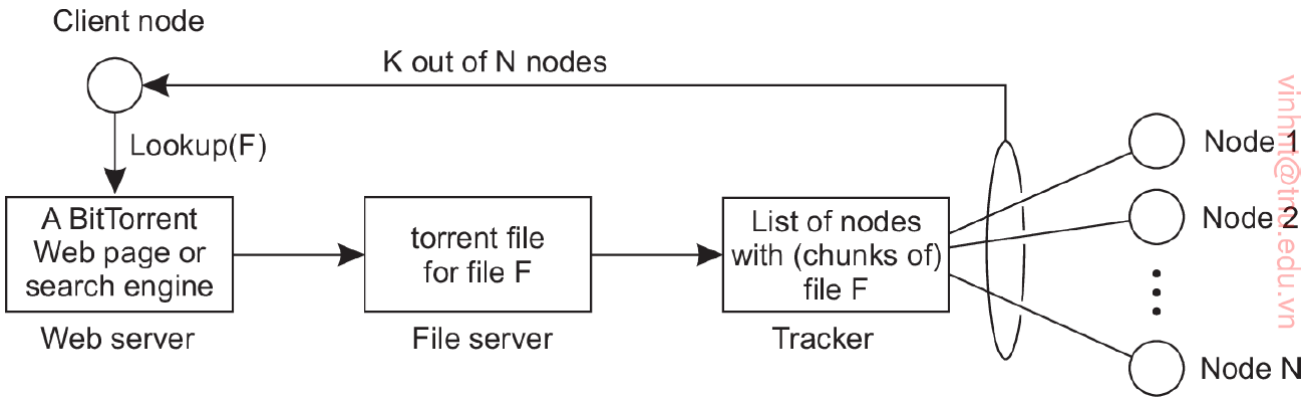
Figure 19: Viewing the Internet as consisting of a collection of edge servers.

Edge-Server Systems: Function and Evolution

- Main functions of edge servers:
  - Serve content close to users
  - Apply filtering and transcoding
  - Reduce latency and network load
- Edge servers often cooperate:
  - One server acts as an **origin server**
  - Other edge servers replicate and cache content
- Modern evolution:
  - Core cloud data centers handle computation
  - Edge servers assist with storage and processing
  - Leads to **distributed cloud** and **fog computing**

Collaborative Distributed Systems

- Collaborative distributed systems often use a **hybrid architecture**
- Key challenge: **getting started**
  - Initial coordination uses **client–server** mechanisms
  - After joining, collaboration becomes **fully decentralized**
- Goal: enable large-scale cooperation among independent nodes
- Typical application: **peer-to-peer file sharing**
- Example: **BitTorrent**
  - Peer-to-peer file downloading system
  - Files are divided into **chunks**
  - Peers download chunks from multiple peers
- Strong focus on **collaboration**
- Prevents **free riding**:
  - Downloading requires uploading



vinhnt@tnu.edu.vn

Figure 20: The principal working of BitTorrent [adapted with permission from Pouwelse et al. [2005]]

BitTorrent: Operation and Incentives

- To download a file, a peer:
  - Accesses a **global directory** (Web site)
  - Retrieves a **torrent file**
  - Contacts a **tracker** to find active peers
- Tracker maintains a list of peers downloading the file
- Once connected, peers exchange chunks **directly**
- Collaboration is enforced by incentives:
  - Peers that upload more receive better service
  - Slow or selfish peers are throttled
- Bottleneck: centralized trackers
- Modern BitTorrent:
  - Uses **DHTs** to distribute tracking
  - Tracker often used only for initial bootstrapping

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Network File System (NFS): Overview

- NFS is a widely used **distributed file system**, originally by Sun Microsystems
- Enables file sharing across a network, mainly in Unix/Linux systems
- Focus on:
  - **NFSv3**: widely deployed
  - **NFSv4**: modern version with enhanced features
- Key goal:
  - Provide a **standardized, transparent view** of remote file systems
  - Hide differences in local file-system implementations

File Access Models in Distributed File Systems

- **Remote access model** (used by NFS):
  - Files remain on the server
  - Clients perform operations remotely (read, write, etc.)
  - File location is transparent to clients
- **Upload/Download model** (contrast):
  - Client downloads entire file, modifies locally, uploads back
  - Example: FTP
- NFS follows the **remote file service model**

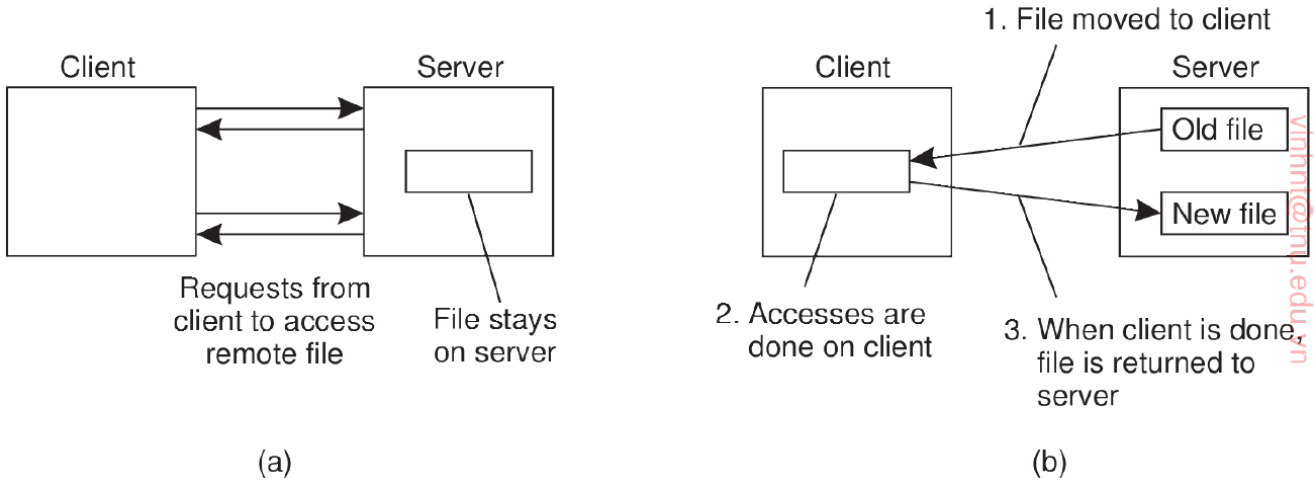


Figure 21: (a) The remote access model. (b) The upload/download model.

NFS Layered Architecture

- NFS uses a **layered architecture** on Unix systems
- Applications issue standard file-system calls
- Calls are handled by the **Virtual File System (VFS)**
- VFS:
  - Hides differences between file systems
  - Forwards operations to:
    - Local file system, or
    - NFS client (for remote files)

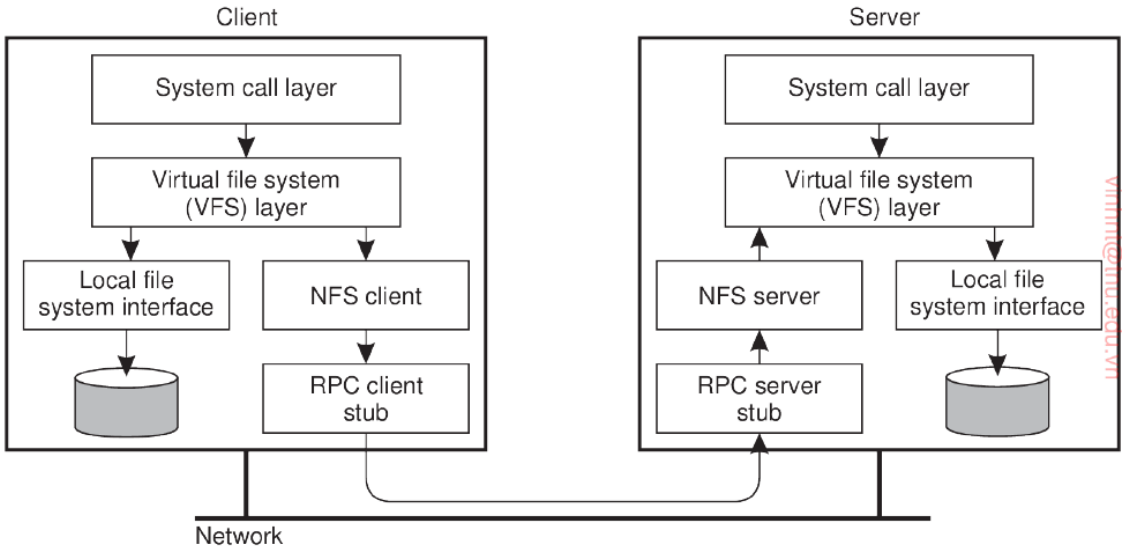


Figure 22: The basic NFS architecture for Unix systems

Client–Server Communication in NFS

- All NFS communication uses **Remote Procedure Calls (RPC)**
- RPC allows a client to invoke procedures on a remote server
- NFS client:
  - Converts file operations into RPC requests
- Server-side:
  - RPC layer translates requests into VFS operations
  - VFS accesses the local file system
- NFS is largely **independent of local file-system types**

NFSv3

- **Stateless server design:** The server maintains no client state; each request is handled independently.
- **Simplicity and robustness:** Server crashes are easy to recover from since no state is preserved.
- **Operation-oriented access:** Files are accessed directly using read and write without open or close.
- **Weaker consistency model:** Limited coordination among concurrent clients leads to weaker consistency.
- **Primary goal:** Achieve scalability and fault tolerance through simplicity.

NFSv4

- **Stateful server design:** The server maintains client state such as open files, locks, and sessions.
- **POSIX-oriented semantics:** Explicit support for open and close makes behavior closer to local file systems.
- **Strong consistency guarantees:** Better synchronization ensures consistent access across clients.
- **Global namespace vision:** A unified file-system view allows automatic traversal across mount points.
- **Primary goal:** Provide correctness and usability through richer semantics.

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

The Web: Evolution and Architecture

- The Web is a large-scale **distributed system**
- Its architecture is similar to classic **client–server systems**
- Originally (1990s), the Web supported:
  - Static and passive documents
- Over time, the Web evolved to support:
  - Dynamically generated content
  - Interactive applications
  - Services instead of only documents
- Core components of a Web system:
  - Web server
  - Web browser (client)

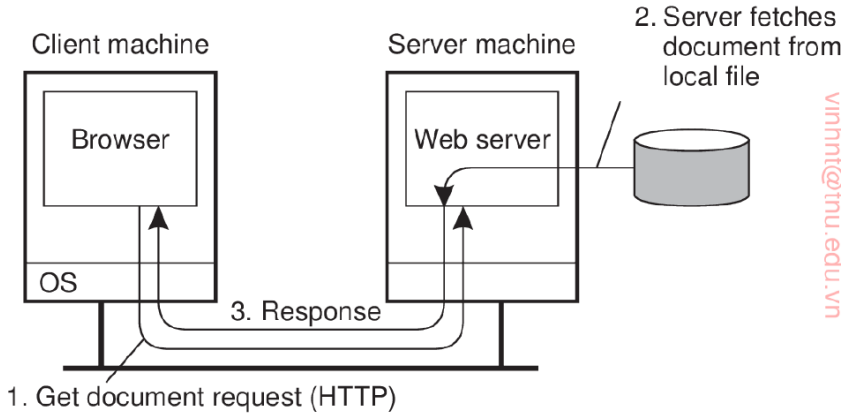


Figure 23: The overall organization of a traditional Web site

Accessing Web Documents

- Web documents are identified using **Uniform Resource Locators (URLs)**
- A URL specifies:
  - The DNS name of the server
  - The location of the document or resource
  - The application-level protocol (e.g., HTTP)
- Clients interact with Web servers through a **browser**
- Browser–server communication follows:
  - The **HTTP** request–response model
- The browser fetches documents and displays them to the user

Web Documents and Client-Side Execution

- The simplest Web documents are plain text files:
  - Transferred verbatim
  - Displayed directly by the browser
- Most Web documents use **HTML**:
  - Markup specifies how content should be displayed
  - Browser interprets and renders markup
- Web documents may embed programs:
  - Typically written in **JavaScript**
  - Executed by the browser
- Embedded scripts enable:
  - Rich user interaction
  - Client-side computation
  - Sending data back to the server

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Multitiered Web Architectures

- Early Web systems followed a simple **two-tier client–server model**
- Client:
  - Web browser
- Server:
  - Stores and serves documents
- Modern Web systems:
  - Generate content dynamically
  - Compose documents on-the-fly from databases and scripts

Server-Side Interaction Using CGI

- CGI (Common Gateway Interface) enables **server-side programs**
- User interaction via:
  - HTML forms
- Processing flow:
  - User submits data
  - Server invokes a CGI program
  - Program processes input (often using a database)
  - Generated HTML is returned to the browser
- Server delegates document generation to external programs

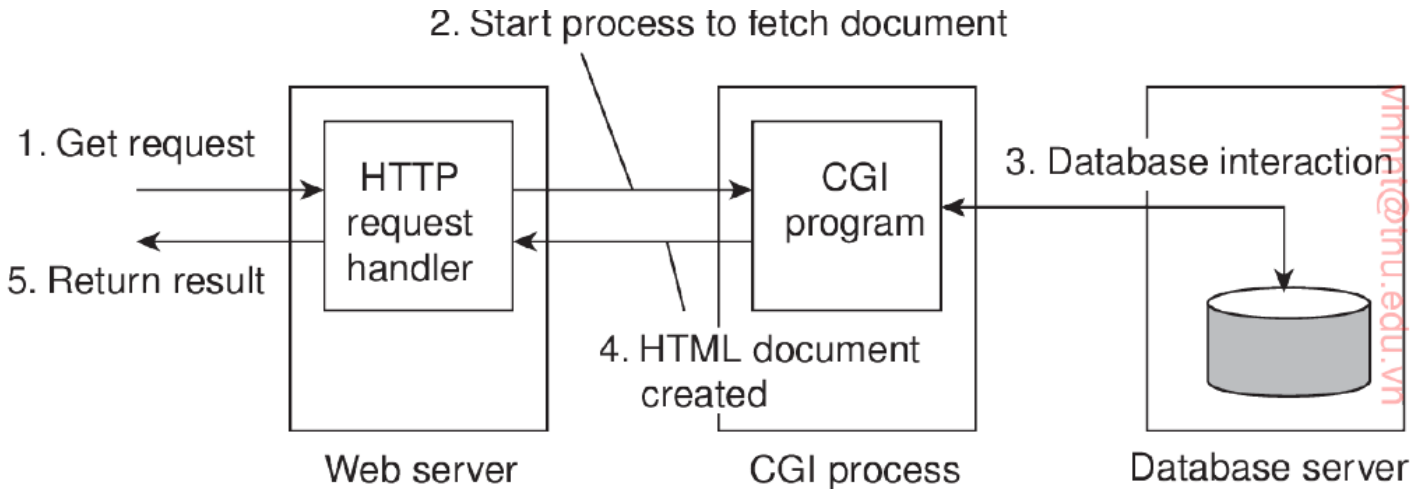


Figure 24: The principle of using server-side CGI programs.

Server-Side Scripts and Dynamic Content

- Modern servers execute **server-side scripts**
- Scripts are:
  - Executed on the server
  - Replaced by their output in the document
- Examples:
  - PHP
  - Server-side Java / Python
- Enables:
  - Personalization
  - Database-driven pages
  - Fully dynamic Web applications

Table of Contents

1. Architectures
2. Architectural Styles
   - Layered Architectures
   - Object-Based Architectures
   - Resource-Based Architectures
   - Publish–Subscribe Architectures
3. Middleware organization
   - Wrappers
   - Interceptor
   - Modifiable Middleware
4. System Architecture
   - Centralized Organizations
   - Decentralized Organizations: Peer-to-Peer Systems
   - Hybrid Architectures
     - Edge-Server Systems
     - Collaborative Distributed Systems
5. Example Architecture
   - Network File System (NFS)
   - The Web
   - Multitiered Web Architectures
6. Summary

Chapter Summary: Architectural Foundations

- Distributed systems can be organized in many different ways
- A key distinction is made between:
  - **Software architecture** – logical organization of components
  - **System architecture** – physical placement of components on machines
- Architectural organization is guided by **architectural styles**
- An architectural style defines:
  - How components interact
  - How responsibilities are separated
  - How independence and reuse are achieved
- Important architectural styles discussed include:
  - Layered architectures
  - Object-based architectures
  - Resource-based (REST) architectures
  - Event-based and publish–subscribe architectures

Chapter Summary: System Organizations

- Distributed systems can be organized as:
  - **Client–server systems** (centralized)
  - **Peer-to-peer systems** (decentralized)
  - **Hybrid systems** combining both approaches
- In client–server architectures:
  - Clients request services
  - Servers process requests and return results
  - Physical distribution follows logical modularization
- In peer-to-peer systems:
  - All processes play equal roles
  - Processes form an overlay network
  - Overlays may be structured or unstructured
- Hybrid architectures combine centralized coordination with decentralized execution